Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación
Ingeniería en Sistemas de Información

Entregable 3: Configuración técnica de servicios y contenedores
Sistema de monitoreo perimetral urbano para la Municipalidad de San Juan Sacatepéquez

[Nombre del integrante 1] [Carné]
[Nombre del integrante 2] [Carné]
[Nombre del integrante 3] [Carné]

Arquitectura de Computadores II, sección 040
[Nombre del catedrático]
31 de octubre de 2026

Índice

Configuración técnica de servicios y contenedores 1
Propósito del documento 1
Organización del repositorio 1
Docker Compose 2
Imágenes propias 3
Variables de entorno 4
Configuración por componente 5
Puesta en marcha y operación 9
Verificación 10
Referencias 11

Configuración técnica de servicios y contenedores

Propósito del documento

Este documento describe la configuración técnica de todos los servicios y contenedores del sistema de monitoreo: cómo está organizado el repositorio, cómo se define cada servicio en Docker Compose, qué imágenes y versiones usa, cómo se configuran Kafka, NGINX, PostgreSQL, Redis, Grafana, Elasticsearch y Airflow, qué variables de entorno existen y cómo se pone en marcha y se verifica el sistema. Corresponde al tercer entregable de la sección 8 del enunciado. Toda la configuración está versionada en el repositorio https://github.com/nglVlr/arqui2, de modo que el sistema se reconstruye idéntico en cualquier máquina con Docker.

Organización del repositorio

Tabla 1

Estructura del repositorio

Carpeta o archivo	Contenido
docker-compose.yml	Definición de todos los servicios, perfiles, redes y volúmenes
.env.example	Plantilla de variables de entorno (el archivo .env real no se versiona)
config/sectores.json	Los siete sectores del municipio: coordenadas, peso, radio, zona escolar y límite de velocidad
infra/nginx/nginx.conf	Balanceador de carga
infra/kafka/crear-topics.sh	Creación de los seis topics
infra/postgres/init/	Esquema, usuarios y carga de sectores de PostgreSQL
infra/grafana/	Datasources, proveedor de dashboards y los cuatro dashboards en JSON
servicios/ingestor, correlador, despachador, aseguradora_mock	Código, Dockerfile y requirements.txt de cada servicio propio
simulador/	Scripts de simulación de eventos (Entregable 4)
airflow/dags/	Tareas periódicas
pruebas/	Plan de JMeter y scripts de verificación de Grafana

Docker Compose

Un solo archivo docker-compose.yml define el sistema completo. Se aplicaron las mismas reglas a todos los servicios: versiones fijas de imagen (nunca latest), límite de memoria (mem_limit), reinicio automático (restart: unless-stopped) en los servicios de larga duración, chequeos de salud (healthcheck) en los servicios de datos y dependencias por estado sano (depends_on con condition: service_healthy), de modo que ningún servicio arranca antes que aquello que necesita. Los servicios se agrupan en perfiles: core (desarrollo), full (agrega Elasticsearch y Airflow) y simulador (a demanda).

Tabla 2

Configuración de cada servicio

Servicio	Imagen	Perfil	Memoria	Puertos	Depende de
kafka	apache/kafka:4.3.1	core, full	1.5 GB	127.0.0.1:9094	-
kafka-init	apache/kafka:4.3.1	core, full	512 MB	-	kafka (sano)
kafka-ui	kafbat/kafka-ui:v1.5.0	core, full	768 MB	8085→8080	kafka (sano)
postgres	postgres:16.15	core, full	1 GB	127.0.0.1:5432	-
redis	redis:7.4.11	core, full	512 MB	127.0.0.1:6379	-
ingestor-1/2/3	monitoreo-sjs/ingestor:1.0	core, full	384 MB	-	kafka (sano), kafka-init (terminado)
nginx	nginx:1.30.5-alpine	core, full	128 MB	8080→80	ingestores
correlador	monitoreo-sjs/correlador:1.0	core, full	512 MB	-	kafka-init, postgres, redis
despachador	monitoreo-sjs/despachador:1.0	core	256 MB	-	kafka-init, postgres, aseguradora
despachador-es	monitoreo-sjs/despachador:1.0	full	256 MB	-	lo mismo + elasticsearch (sano)
aseguradora-mock	monitoreo-sjs/aseguradora-mock:1.0	core, full	128 MB	-	-
grafana	grafana/grafana:12.4.12	core, full	512 MB	3000→3000	postgres (sano)
elasticsearch	elasticsearch:8.19.22	full	2 GB	127.0.0.1:9200	-
airflow	apache/airflow:3.3.2-python3.12	full	3 GB	8090→8080	postgres (sano)
simulador	monitoreo-sjs/simulador:1.0	simulador	512 MB	-	-

Nota. Todos los servicios de larga duración usan restart: unless-stopped; kafka-init usa restart: no porque termina después de crear los topics.

Los tres ingestores comparten la misma definición mediante un ancla de YAML (&ingestor) y solo cambian su identificador (Figura 1). Ninguno publica puertos: solo se llega a ellos a través de NGINX.

Figura 1

Definición del ingestor en docker-compose.yml

```
  # Tres copias del mismo código; solo cambia INGESTOR_ID.
  # No publican puertos: solo se llega a ellos a través de NGINX.
  ingestor-1: &ingestor
    build: ./servicios/ingestor
    image: monitoreo-sjs/ingestor:1.0
    profiles: ["core", "full"]
    restart: unless-stopped
    mem_limit: 384m
    environment:
      INGESTOR_ID: ingestor-1
      KAFKA_BOOTSTRAP: kafka:9092
      SECTORES_PATH: /config/sectores.json
      TZ: ${TZ}
    volumes:
      - ./config:/config:ro
    depends_on:
      kafka:
        condition: service_healthy
      kafka-init:
        condition: service_completed_successfully
```

Nota. Fragmento del archivo docker-compose.yml, líneas 128 a 147.

Tabla 3

Chequeos de salud

Servicio	Comando	Intervalo	Reintentos
kafka	kafka-broker-api-versions.sh --bootstrap-server localhost:9092	10 s	12 (espera inicial 20 s)
postgres	pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}	5 s	12
redis	redis-cli ping	5 s	12
elasticsearch	curl a /_cluster/health y espera estado green o yellow	10 s	30 (espera inicial 30 s)

Imágenes propias

Los cinco servicios propios (ingestor, correlador, despachador, aseguradora simulada y simulador) se construyen a partir de python:3.12.15-slim con un Dockerfile corto. Las dependencias se instalan antes de copiar el código para que Docker reutilice

esa capa cuando solo cambia el código (Figura 2). Cada servicio tiene su propio requirements.txt con versiones fijas (Tabla 4).

Figura 2

Dockerfile del ingestor

```
# Imagen del ingestor: Python 3.12 + FastAPI servido con uvicorn.
FROM python:3.12.15-slim

WORKDIR /app
ENV PYTHONUNBUFFERED=1

# Primero las dependencias: Docker reutiliza esta capa si solo cambia el código
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY *.py .

EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000", "--no-access-log"]
```

Nota. Fragmento del archivo servicios/ingestor/Dockerfile, líneas 1 a 14.

Tabla 4

Dependencias de Python por servicio

Servicio	Librerías y versiones
ingestor	fastapi 0.142.2, uvicorn 0.54.0, pydantic 2.13.5, confluent-kafka 2.15.1
correlador	confluent-kafka 2.15.1, redis 7.4.1, psycopg2-binary 2.9.13
despachador	confluent-kafka 2.15.1, psycopg2-binary 2.9.13, requests 2.34.2
aseguradora-mock	fastapi 0.142.2, uvicorn 0.54.0
simulador	requests 2.34.2, redis 7.4.1, psycopg2-binary 2.9.13

Variables de entorno

Las contraseñas, la clave de las placas y los parámetros del correlador se leen del archivo .env, que no se sube al repositorio. Se entrega .env.example como plantilla; el manual de instalación (README) incluye un comando que crea el .env con contraseñas aleatorias.

Tabla 5

Variables de entorno del sistema

Variable	Valor de ejemplo	Uso
TZ	America/Guatemala	Zona horaria de los contenedores
POSTGRES_DB / POSTGRES_USER / POSTGRES_PASSWORD	monitoreo / admin / (secreto)	Base de datos y usuario administrador
APP_DB_USER / APP_DB_PASSWORD	app / (secreto)	Usuario de lectura y escritura de los servicios
GRAFANA_RO_PASSWORD	(secreto)	Usuario grafana_ro, solo lectura
GRAFANA_ADMIN_USER / GRAFANA_ADMIN_PASSWORD	admin / (secreto)	Acceso a la interfaz de Grafana
AIRFLOW_ADMIN_PASSWORD	(secreto)	Acceso a la interfaz de Airflow
PLATE_HMAC_KEY	(64 caracteres hexadecimales)	Clave del HMAC de las placas
CORRELATION_WINDOW_SECONDS	300 (60 en la demostración)	Ventana de correlación
ACOUSTIC_MIN_PROBABILITY	0.7	Umbral de la regla de ataque armado
SLO_LATENCIA_MS	2000	Umbral de latencia de alertas
SCHOOL_HOURS	06:30-08:00,12:00-13:30	Horario escolar

Configuración por componente

Kafka

Kafka 4.3 corre en modo KRaft, con un solo nodo que es a la vez broker y controlador. Tiene dos listeners: INTERNO (kafka:9092) para los contenedores y EXTERNO (localhost:9094) solo para depurar desde el servidor. La creación automática de topics está desactivada (KAFKA_AUTO_CREATE_TOPICS_ENABLE=false), la retención es de 168 horas y, por ser un solo broker, los factores de replicación internos son 1. El contenedor kafka-init crea los topics con el script de la Figura 3.

Figura 3

Script de creación de topics

```
set -e

SERVIDOR="kafka:9092"
RETENCION_MS=604800000   # 7 días en milisegundos

crear_topic() {
  nombre=$1
  particiones=$2
  /opt/kafka/bin/kafka-topics.sh --bootstrap-server "$SERVIDOR" --create --if-not-exists \
    --topic "$nombre" --partitions "$particiones" --replication-factor 1 \
    --config retention.ms=$RETENCION_MS
  echo "Topic listo: $nombre ($particiones particiones)"
}

# 3 particiones donde hay más volumen, para poder repartir el trabajo por zona
crear_topic events.standardized 3
crear_topic events.invalid 1
crear_topic correlated.alerts 3
crear_topic external.pnc 1
crear_topic external.bomberos 1
crear_topic external.conred 1
```

Nota. Fragmento del archivo infra/kafka/crear-topics.sh, líneas 7 a 27.

NGINX

NGINX reparte las peticiones en turnos entre los tres ingestores. Si uno falla dos veces seguidas lo deja de usar 10 segundos y reintenta la petición en otro (Figura 4). Con zone y resolve, vuelve a resolver la dirección de un ingestor cuando su contenedor se reinicia, y agrega la cabecera X-Upstream con el ingestor que atendió.

Figura 4

Configuración del balanceador

```
    upstream ingestores {
        # zone + resolve: NGINX vuelve a preguntar la IP de cada ingestor
        # cuando su contenedor se reinicia (Docker puede darle otra IP)
        zone ingestores 64k;
        server ingestor-1:8000 max_fails=2 fail_timeout=10s resolve;
        server ingestor-2:8000 max_fails=2 fail_timeout=10s resolve;
        server ingestor-3:8000 max_fails=2 fail_timeout=10s resolve;
        keepalive 64;   # reutiliza conexiones hacia los ingestores
    }

    server {
        listen 80;
        client_max_body_size 5m;   # lotes de hasta 500 eventos

        location / {
            proxy_pass http://ingestores;
            proxy_http_version 1.1;
            proxy_set_header Connection "";
            proxy_set_header Host $host;
            proxy_connect_timeout 2s;
            proxy_read_timeout 15s;
            # Si un ingestor no responde, se prueba el siguiente (también en POST)
            proxy_next_upstream error timeout http_502 non_idempotent;
            proxy_next_upstream_tries 3;
            # Header para demostrar el balanceo: dirección del ingestor que atendió
            add_header X-Upstream $upstream_addr always;
        }
    }
```

Nota. Fragmento del archivo infra/nginx/nginx.conf, líneas 20 a 47.

PostgreSQL

Los scripts de infra/postgres/init se ejecutan solos la primera vez que se crea el volumen: 01_esquema.sql crea las nueve tablas con fechas TIMESTAMPTZ e índices por tiempo y sector, 02_roles.sql crea los usuarios app (lectura y escritura) y grafana_ro (solo lectura) leyendo sus contraseñas de las variables de entorno con \getenv, y 03_sectores.sql carga los siete sectores.

Tabla 6

Tablas de la base de datos

Tabla	Contenido	Escribe
zones	Los siete sectores	Script inicial
events	Eventos válidos (data y metadata en JSONB)	Correlador
alerts	Alertas con sus eventos de origen	Correlador
slo_violations	Alertas que superaron el umbral de latencia	Correlador
invalid_events	Eventos rechazados (dead-letter)	Despachador
dispatches	Envíos a instituciones	Despachador
daily_stats	Resumen diario de alertas	Airflow
false_positive_stats	Proporción de falsos positivos por sector	Airflow
evaluation_results	Resultados de la evaluación de escenarios	evaluar.py

Redis

Redis 7.4 se ejecuta con persistencia AOF (redis-server --appendonly yes), de modo que al reiniciarse recupera el estado del correlador: ventanas por sector, vehículos, eventos procesados, cooldowns y la lista de placas buscadas.

Grafana

Grafana 12.4 se configura por archivos (aprovisionamiento): el datasource de PostgreSQL usa el usuario grafana_ro y el de Elasticsearch apunta al índice alertas (Figura 5); un proveedor carga los cuatro dashboards desde infra/grafana/dashboards. Las contraseñas se toman de variables de entorno, no del archivo. El detalle de los dashboards está en el Entregable 6.

Figura 5

Datasources aprovisionados

```
datasources:
  - name: PostgreSQL
    uid: postgres-sjs
    type: grafana-postgresql-datasource
    url: postgres:5432
    user: grafana_ro
    isDefault: true
    editable: false
    jsonData:
      database: $POSTGRES_DB
      sslmode: disable
      postgresVersion: 1600
      timescaledb: false
      maxOpenConns: 10
    secureJsonData:
      password: $GRAFANA_RO_PASSWORD

  - name: Elasticsearch
    uid: elastic-sjs
    type: elasticsearch
    url: http://elasticsearch:9200
    editable: false
    jsonData:
      index: alertas
      timeField: createdAt
```

Nota. Fragmento del archivo infra/grafana/provisioning/datasources/datasources.yml, líneas 9 a 33.

Elasticsearch y Airflow

Elasticsearch 8.19 corre como nodo único (discovery.type=single-node), con 1 GB de memoria de Java y la seguridad desactivada solo por tratarse de un laboratorio en una sola máquina, con su puerto publicado únicamente en 127.0.0.1. Airflow 3.3 corre en modo standalone: antes de arrancar, el contenedor escribe la contraseña del usuario admin desde AIRFLOW_ADMIN_PASSWORD; los DAGs se montan en solo lectura desde airflow/dags, y los ejemplos de Airflow están desactivados.

Puesta en marcha y operación

Con Docker Desktop instalado, el sistema se levanta y se opera con los comandos de la Figura 6. La primera vez se descargan unos 5 GB de imágenes.

Figura 6

Comandos de puesta en marcha y operación

```
git clone https://github.com/nglVlr/arqui2.git
cd arqui2
# crear .env con el comando del README (paso 2)
docker compose --profile full up -d --build      # levantar todo
docker compose ps                                # estado de los servicios
docker compose run --rm simulador cargar_buscados.py
docker compose logs -f correlador                # alertas en vivo
docker stats                                     # CPU y memoria
docker compose --profile full down               # detener (conserva datos)
docker compose --profile full down -v            # detener y borrar datos
```

Nota. Comandos documentados en el README del repositorio.

Verificación

La configuración se verificó de tres formas: clonando el repositorio en una carpeta nueva y levantando el sistema desde cero siguiendo solo el manual (todos los servicios iniciaron y los escenarios generaron sus alertas); revisando con docker compose ps que los servicios con chequeo de salud aparezcan como healthy; y ejecutando el script pruebas/grafana/validar-dashboards.ps1, que corre las 30 consultas de los cuatro dashboards contra Grafana (30 correctas, 0 errores).

Referencias

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. En *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*.

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal, 2014*(239), Artículo 2.